

Krakken-Disk

v4.6.0 — Butterfly Edition

A Post-Quantum Encrypted Disk Manager for Linux

User & Technical Manual

Kyber-1024 / X448 Hybrid KEM
2048-bit Krakken-2048 Butterfly Permutation
AVX2-Accelerated • Plausible Deniability

Jean-Francois Lachance-Caumartin (Effjy)

Lead Cryptographer & GUI Developer

Licensed under the MIT License

June 2, 2026

Contents

1	Introduction	2
1.1	Key Highlights	2
2	Cryptographic Architecture	2
2.1	The Krakken Abyssal Permutation	3
2.2	Layer 6 — Butterfly Diffusion (XRBD)	3
2.3	Layer 7 — PRESSURE (ARX Mixing)	4
2.4	Layer 9 — InkCloud Shuffle	4
2.5	High-Performance AEAD Stream Encryption	4
3	Prerequisites	4
3.1	Build System & Compilers	4
3.2	Required Libraries	4
3.3	Graphical User Interface (GTK)	4
3.4	Install Dependencies (Debian / Ubuntu)	5
4	Compilation & Installation	5
4.1	Step 1 — Dependency Validation	5
4.2	Step 2 — Compilation	5
4.3	Step 3 — Installation	5
4.4	Step 4 — Uninstallation	5
5	Usage	5
5.1	Launching the Application	5
5.2	Creating a Secure Container	6
5.3	Accessing (Mounting) an Existing Volume	6
6	Security Best Practices	6
7	Authors & License	6

1 Introduction

KRAKKEN-DISK is an ultra-secure, high-performance encrypted disk manager engineered specifically for the post-quantum era. Powered by the 2048-bit **Krakken-2048 Butterfly** permutation, it provides a uniform 256-bit post-Grover security margin across all volume layers. By combining lattice-based cryptography, elliptic-curve cryptography, and hardware-accelerated AVX2 SIMD architectures, KRAKKEN-DISK keeps your data private even against future quantum-computing adversaries.

The *Butterfly Edition* refines the original Krakken-Disk design by replacing the permutation core with the Krakken-2048 Butterfly variant, whose novel XOR-Rotation Butterfly Diffusion (XRBD) layer achieves full word-level avalanche in a single pass. This allows the round count to be reduced from ten to eight while improving security margins.

1.1 Key Highlights

- **Post-Quantum Security Margin.** A native 2048-bit wide-state permutation provides a uniform 256-bit post-Grover security margin across the header and data layers.
- **Hybrid Key Encapsulation (KEM).** Combines post-quantum **Kyber-1024** (lattice-based) with classical **X448** (elliptic-curve Diffie-Hellman) to secure master and file keys.
- **Abyssal Permutation Core.** Hand-tuned AVX2 SIMD vectorizations with register-only MDS mixing and column-pressure steps for maximum performance on modern CPUs.
- **Plausible Deniability.** Full IND-RND compliance: volumes carry no identifiable headers, signatures, or metadata blocks, making them mathematically indistinguishable from random data.
- **Anti-Brute-Force Protection.** **Argon2id** key derivation locked to 1 GB of RAM renders GPU- and ASIC-based brute-force attacks economically and computationally impractical.
- **Dual-Generation Compatibility.** Seamless trial-decryption supports legacy V3 volumes (XChaCha) and next-generation V4 volumes (Krakken-2048).
- **FUSE 3 Mounting.** Exposes encrypted containers as transparent, read-write filesystem directories in user space.

2 Cryptographic Architecture

KRAKKEN-DISK employs a multi-tiered cryptographic design to protect files from physical and quantum adversaries. The key hierarchy flows from the user password down to the per-sector encryption keys as follows:

1. The **user password** is stretched by the **Argon2id** KDF (locked to 1 GB of RAM) into a 512-bit **master key**.
2. The master key drives a **Krakken-2048 duplex** construction that unwraps the **hybrid private keys** (Kyber-1024 + X448).
3. **Hybrid decapsulation** of those keys yields a 256-bit **file key**.
4. The file key populates a **locked sector cache**, from which individual sectors are encrypted and decrypted using the **Krakken-2048 sponge**.

Design rationale

The hybrid KEM ensures that breaking *either* the lattice scheme *or* the elliptic-curve scheme alone is insufficient to recover keys: an attacker must defeat both simultaneously.

2.1 The Krakken Abyssal Permutation

The permutation operates over a 2048-bit state, organized as 32×64 -bit lanes arranged in a 4×8 column grid. The Butterfly Edition executes **8 rounds**, each composed of nine distinct layers applied in sequence.

Layer	Name	Type	Description
L1	Θ (Theta)	SPN	Column parity mixing
L2	Σ (Sigma)	SPN	Circulant MDS over $\text{GF}(2^8)$
L3	ρ (Rho)	SPN	Per-word rotation
L4	Π (Pi)	SPN	Column permutation
L5	χ (Chi)	SPN	ABYSSAL S-box (cross-coupled pairs)
L6	Butterfly Diffusion (XRBD)	Novel	5-stage XOR-rotation butterfly network
L7	PRESSURE	ARX	Modular addition with XOR-shift mixing
L8	ι (Iota)	Key Schedule	Round constant addition (per-word)
L9	InkCloud	Diffusion	Global word permutation + rotation

Table 1: The nine layers of a single Krakken-2048 Butterfly round.

2.2 Layer 6 — Butterfly Diffusion (XRBD)

The XOR-Rotation Butterfly Diffusion layer is the central architectural novelty of Krakken-2048. It mixes all 32 words of state so that every output word depends on every input word after a single pass, using only word-level XOR and rotation operations.

Definition. Let r_0, r_1, r_2, r_3, r_4 be the rotation constants (13, 23, 37, 41, 53). For stage $k \in \{0, 1, 2, 3, 4\}$, let $\delta_k = 2^k \in \{1, 2, 4, 8, 16\}$. For all $i \in \{0, 1, \dots, 31\}$ such that $(i \text{ AND } \delta_k) = 0$ (bitwise AND), execute:

$$\begin{aligned} \text{state}[i] &\leftarrow \text{state}[i] \oplus \text{state}[i \oplus \delta_k], \\ \text{state}[i \oplus \delta_k] &\leftarrow \text{state}[i \oplus \delta_k] \oplus \text{RotL}(\text{state}[i], r_k). \end{aligned}$$

Key properties.

- **Bijective.** Each crossover step is invertible (Theorem 1 in the paper).
- **Complete dependency.** Achieves $\log_2(32) = 5$ stages for full word-level mixing (Theorem 2).
- **Optimal.** Matches the theoretical lower bound for pairwise-mixing networks (Theorem 3).
- **Rotation-enhanced.** Asymmetric rotation constants break bit-aligned symmetries.

2.3 Layer 7 — PRESSURE (ARX Mixing)

After Butterfly Diffusion connects all 32 words, PRESSURE introduces modular-arithmetic non-linearity via carry propagation that crosses byte boundaries. For each column $c \in [8]$ with words (a, b, cc, d) :

$$\begin{aligned} a &\leftarrow a + (cc \oplus (cc \gg 17)), \\ b &\leftarrow b + (d \oplus (d \gg 17)), \\ cc &\leftarrow cc + (a \oplus (a \ll 31)), \\ d &\leftarrow d + (b \oplus (b \ll 31)), \end{aligned}$$

followed by the post-rotations $b \leftarrow \text{RotL}(b, 7)$ and $d \leftarrow \text{RotL}(d, 19)$.

2.4 Layer 9 — InkCloud Shuffle

The final step of each round is a global word permutation combined with rotation:

$$\text{state}'[(7i) \bmod 32] \leftarrow \text{RotL}(\text{state}[i], 11), \quad i \in [32].$$

The multiplier 7 (coprime to 32) generates a full 32-cycle, ensuring every word returns to its original position only after 32 rounds.

2.5 High-Performance AEAD Stream Encryption

For file operations, KRAKKEN-DISK divides streams into fixed 4MB segments. Each segment is processed independently and in parallel using a thread pool (up to 8 hardware threads), each with its own sponge state:

$$\text{keystream}_i = \text{Krakken-Sponge}(\text{FileKey} \parallel \text{Nonce} \parallel \text{LE64}(i)).$$

A BLAKE2b-256 MAC is computed over the entire stream for ciphertext authentication.

3 Prerequisites

To compile and run KRAKKEN-DISK on Linux, ensure the following packages are installed.

3.1 Build System & Compilers

- **GCC** — with AVX2 instruction-set support and C11/GNU11 compatibility
- **GNU Make**
- **pkg-config**

3.2 Required Libraries

- **libsodium** — cryptographic primitives
- **libcrypto** — OpenSSL EVP for X448 scalar multiplication
- **FUSE 3** (`libfuse3-dev` / `fuse3`) — virtual filesystem interface

3.3 Graphical User Interface (GTK)

- **GTK 3** or **GTK 4** development libraries — the modern dark-themed graphical dashboard
- **ncurses** — automatic fallback to a terminal UI when no GUI environment is available

3.4 Install Dependencies (Debian / Ubuntu)

```
sudo apt update
sudo apt install build-essential pkg-config libsodium-dev \
    libssl-dev libfuse3-dev libgtk-3-dev libncurses5-dev
```

4 Compilation & Installation

4.1 Step 1 — Dependency Validation

Validate that all required tools and libraries are present:

```
make check-deps
```

4.2 Step 2 — Compilation

Build the optimized production binary (automatically detects CPU features and uses AVX2 where available):

```
make
```

To inspect the underlying compiler flags and commands during the build, use verbose mode:

```
make V=1
```

4.3 Step 3 — Installation

Install the application globally. This copies the executable to `/usr/local/bin`, registers the desktop application shortcut, and installs the Krakken logo:

```
sudo make install
```

4.4 Step 4 — Uninstallation

To completely remove KRAKKEN-DISK and its configuration entries from the host system:

```
sudo make uninstall
```

5 Usage

5.1 Launching the Application

If installed globally, launch KRAKKEN-DISK from your desktop applications menu, or invoke it directly:

```
krakken-disk
```

Alternatively, run it from the build directory:

```
make run
```

5.2 Creating a Secure Container

1. Enter the target output path for the volume in the **Encrypted Volume File** box.
2. Specify the size of the volume in megabytes (minimum 10 MB, maximum 1 TB).
3. Type a strong passphrase in the **Credentials** card.
4. Click **Create Volume**. A progress bar reflects the structural creation and formatting of the virtual filesystem.

5.3 Accessing (Mounting) an Existing Volume

1. Click the **Browse** folder icon to select your encrypted volume file.
2. Type the password in the **Credentials** pane.
3. Click **Open**. The status panel confirms the trial-decryption status.
4. Click **Mount** and select a target folder. The FUSE daemon runs in the background, mounting the volume.
5. Read, write, copy, and modify files within that folder as normal.
6. When finished, click **Unmount** and **Close** to flush all modifications to disk and lock the cryptographic keys.

6 Security Best Practices

Unencrypted Swap Partition Alert

On startup, KRAKKEN-DISK inspects `/proc/swaps` to determine whether unencrypted swap memory is active. If detected, it displays a security warning. Unencrypted swap can write active memory pages containing keys or plaintexts to persistent storage, compromising security. It is strongly recommended to disable swap (`sudo swapoff -a`) or encrypt it using LUKS.

Memory Locking (mlock)

KRAKKEN-DISK attempts to call `sodium_mlock` on all sensitive key containers, file handles, and cache sectors to prevent them from being paged out to disk. To enable this, ensure your user shell has sufficient resource limits, or run the program with elevated privileges.

7 Authors & License

- **Lead Cryptographer & GUI Developer:** Jean-Francois Lachance-Caumartin (Effjy)
- **Contact:** effjy@protonmail.com
- **Repository:** github.com/effjy/krakken-disk-butterfly

This project is licensed under the MIT License. See the **LICENSE** file in the repository for the full terms.